Fase 2 – De la Incertidumbre al Diseño: Dossier de Pre-diseño

Felipe Mejía Alvarado

Laura Natalia Olmos Parra

Laura Valentina Ospina Rengifo

Laura Palacios Ochoa

Facultad de Ingeniería, Universidad El Bosque

Ing. Samuel Agustín Galindo Pinzón

Bogotá D.C, Colombia

15 de octubre de 2026

Tabla de Contenido

D. Ejecución de Prototipos 3

D.1 Prototipo de lógica y estructura técnica 3

D.2 Prototipo de interacción con el usuario 4

E. Ficha de Patrón: Strategy en el Motor de Generación 6

F. Principios SOLID y Prueba de Cambio 7

G. Escalabilidad 9

H. Primer Borrador de Diagramas 10

I. Qué Cambió y Ruta hacia el Entregable 2.3 12

Referencias 14

Anexo A. Evidencia del Prototipo Técnico 15

Anexo B. Prueba de Cambio: Código de la Nueva Regla 18

Anexo C. Evidencia del Prototipo de Interacción 19

Anexo D. Bitácora de Trabajo 20

Anexo E. Memorias Individuales 21

Laura Natalia Olmos Parra 21

Laura Valentina Ospina Rengifo 22

Laura Palacios Ochoa 23

Felipe Mejía Alvarado 23

Anexo F. Autoevaluación del Equipo 24

Anexo G. Transparencia de Uso de IA 24

D. Ejecución de Prototipos

D.1 Prototipo de lógica y estructura técnica

Pregunta (Fase 1). ¿Puede una versión mínima del modelo de datos y de la búsqueda con retroceso (backtracking) generar un horario completo que cumpla todas las reglas obligatorias, y reconocer cuándo no existe ese horario?

Se construyó el motor del sistema en Python, todavía sin la aplicación web ni la base de datos del MVP. Revisa los datos, aplica las reglas obligatorias y busca el horario eligiendo primero la clase con menos opciones y descartando de antemano las opciones bloqueadas, con 84 clases, 12 docentes, 8 aulas y 32 franjas de dos horas. Para verlo funcionar, un script muestra sus resultados en una página, incluida la construcción del horario paso a paso (Anexo A, Figuras 3 a 5).

Tabla 1

Resultados del prototipo técnico frente a su criterio de éxito

Escenario	Resultado esperado	Resultado obtenido	¿Cumple?
Horario posible, tamaño actual (10 casos)	Horario completo sin incumplir ninguna regla	10 de 10 completos y sin incumplimientos, según un verificador independiente; 0,14 s en promedio	Sí
Un docente con 11 clases y solo 4 franjas disponibles	Avisar que no hay horario posible y por qué	La revisión previa de datos lo avisa en menos de 0,5 s y nombra al docente y su falta de disponibilidad	Sí
El grupo 7A con dos docentes que solo dictan de noche (14 clases para 10 franjas nocturnas)	Avisar que no hay horario posible y por qué	Primera versión: la búsqueda se detiene a los 60 s sin horario y culpa a uno de los docentes, que no es la causa real	No
El mismo caso con la revisión previa mejorada	Igual	Avisa en menos de 0,5 s que el grupo 7A necesita 14 franjas y	Sí

		sus docentes solo están disponibles en 10	
Una sola sala de cómputo con el doble de cursos	Igual	Avisa en menos de 0,5 s que 60 clases requieren computadores y solo hay 32 espacios disponibles	Sí

Cuando existe un horario posible, sí se cumple el criterio de éxito. Cuando no existe, a medias: el sistema nunca presentó un horario que rompiera una regla, como exige el MVP, pero en el caso del grupo 7A tardó 60 s, muy por encima del máximo de 5 s, y no explicó la causa real, como exige el requisito de reportar los conflictos con las reglas y elementos involucrados. Por eso se agregó a la revisión previa una comprobación por grupo (sus clases no pueden superar las franjas en que sus docentes están disponibles), que reporta la causa al instante.

D.2 Prototipo de interacción con el usuario

Pregunta (Fase 1). ¿Puede un coordinador identificar la causa de un conflicto, reconocer las clases involucradas y cambiar una de ellas sin crear un nuevo incumplimiento?

Se construyó un prototipo navegable de fidelidad media que sigue el boceto, con un cruce real de docente provocado por el prototipo técnico, que también calcula qué franjas sirven (13 de 31). Tras evaluarlo se hizo una versión 2 que muestra primero las franjas válidas (Anexo C, Figuras 7 y 8).

Cómo se midió. El criterio de la Fase 1 dependía de reunir a tres personas, algo que el enunciado no exige; antes de evaluar, se reemplazó por tres métodos de inspección: un recorrido cognitivo, que revisa paso a paso si el usuario sabría qué hacer y entendería la respuesta; las diez heurísticas de usabilidad de Nielsen (1994); y el modelo de pulsaciones para estimar el tiempo de la tarea (Card et al., 1980). El nuevo criterio: detectar el conflicto, entender su causa y reconocer un rechazo sin fallas, ningún problema catastrófico y la tarea sin errores en menos de 2 minutos.

Tabla 2

Recorrido cognitivo y tiempo estimado de cada paso

Paso	¿Sabe qué hacer y lo encuentra?	¿Entiende la respuesta?	Resultado	Tiempo (s)
1. Notar el conflicto	La clase aparece en rojo y un aviso lateral lo anuncia	El aviso nombra al docente, el día y la hora	Sin falla	6,2
2. Abrir el detalle	Botón «Revisar» dentro del aviso	Se abre el detalle	Sin falla	1,3
3. Entender la causa	El texto nombra al docente, las dos clases, sus grupos y la hora	Muestra las dos clases lado a lado	Sin falla	19,7
4. Elegir una nueva franja	V1: la lista no indica cuáles sirven. V2: las válidas aparecen primero como botones	V1: solo al validar. V2: toda sugerencia es válida	V1: falla parcial. V2: sin falla	4,0 / 10,2
5. Validar y leer la respuesta	Botón «Validar cambio»	Rechazo en rojo con el motivo en lenguaje común, o aceptación en verde	Sin falla	10,1
6. Guardar y comprobar	Botón «Guardar horario»	La clase queda en verde y el aviso dice «Sin conflictos»	Sin falla	5,8

Tabla 3

Problemas encontrados con las heurísticas de Nielsen

Problema	Heurística	Gravedad (0 a 4)	Cambio propuesto para 2.3
La lista muestra las 31 franjas sin distinguir las que sirven; el usuario prueba al azar	Prevención de errores	3 (grave)	Mostrar primero las franjas válidas (hecho en la V2)
En el detalle no se ve la semana; el usuario debe recordar qué franjas tiene libres el grupo	Reconocer en vez de recordar	2	Incluir una vista reducida de la semana
Validar y guardar son dos pasos, con dos botones «Cancelar» que hacen cosas distintas	Consistencia	2	Un solo botón «Aplicar cambio»

No se puede deshacer un cambio ya guardado	Control y libertad	2	Deshacer el último cambio
Los docentes aparecen con un código y no con su nombre	Lenguaje del usuario	1	Mostrar el nombre

¿Se respondió la pregunta? Sí. En la versión 1, detectar el conflicto, entender su causa y reconocer un rechazo no tuvieron fallas, pero elegir la franja sí: escogiendo al azar se falla 1,3 veces en promedio (65 s) y hasta 18 veces en el peor caso (5 minutos). Como el sistema calcula las franjas válidas en 2,9 ms, la versión 2 las muestra primero; al reevaluarla, ningún paso tuvo fallas y la tarea toma 52 s sin ensayo y error. La inspección no reemplaza observar usuarios reales; esa prueba se recomienda con los mockups definitivos.

E. Ficha de Patrón: Strategy en el Motor de Generación

La Tabla 4 sigue la estructura de contexto, problema, fuerzas y solución de Ortega-Arjona (2010).

Tabla 4

Ficha del patrón Strategy aplicado al motor de generación

Elemento	Descripción
Contexto	Hoy el horario se genera con búsqueda con retroceso, pero el M1 prevé usar en el futuro un solucionador de restricciones (por ejemplo, CP-SAT de Google) o búsqueda local. El equipo también necesita comparar métodos con los mismos datos.
Problema	Cómo usar, comparar o reemplazar el método de generación sin modificar las partes que cargan los datos, muestran el horario o validan cambios manuales.
Fuerzas	1) Flexibilidad frente a simplicidad: el MVP solo necesita un método. 2) Contrato común frente a diferencias reales: un solucionador externo puede optimizar y demostrar que no hay solución; la búsqueda local necesita un horario inicial; el retroceso solo encuentra uno válido. 3) Explicaciones detalladas frente a una interfaz mínima: cada método explica sus fallas de forma distinta.
Por qué encaja	Strategy agrupa algoritmos intercambiables detrás de una interfaz común que el cliente usa sin saber cómo funcionan (Gamma et al., 1994). Aquí todos los

	métodos reciben los mismos datos y devuelven un resultado que se puede verificar.
Solución	Una interfaz Solver con la operación resolver. BacktrackingSolver la implementa y ServicioProgramacion recibe el método que debe usar. Todo resultado termina en uno de cuatro estados: horario encontrado, sin solución, tiempo agotado o datos inválidos.
Sin el patrón	El servicio necesitaría un condicional por cada método y conocería todos sus detalles; cada método nuevo obligaría a modificarlo, y comparar métodos exigiría duplicar código.
Con otro patrón	Template Method fijaría un esqueleto de búsqueda con pasos reemplazables. Sirve para variantes del retroceso, pero no para un solucionador externo, que tiene su propia búsqueda interna. Por eso se descarta.
Beneficios y costos	Beneficios: métodos intercambiables y comparables, y pruebas que no dependen del método. Costos: una interfaz y una clase más, y el riesgo de que el resultado común pierda detalles propios de cada método.
Dónde aparece	horarios/solver.py (Solver y BacktrackingSolver) y la Figura 1. ServicioProgramacion y CPSATSolver están propuestos y aún no existen.

Contraste con la hipótesis de la Fase 1. Se confirma con un ajuste. Cambiar de método es una necesidad real: con la mayoría de las aulas ocupadas, ninguna variante del retroceso encontró horario en 30 s (Anexo A). En cambio, las variantes del propio retroceso no justifican estrategias separadas; bastó con parámetros.

F. Principios SOLID y Prueba de Cambio

Abierto/cerrado e inversión de dependencias (Hu, 2023). El motor y el editor de cambios trabajan con la abstracción Restriccion y no con reglas concretas; solo RegistroRestricciones sabe cuáles reglas están activas. Cada regla declara además a qué franjas y a qué clases afecta, para que el motor descarte opciones sin conocerla:

```python
class RestriccionRelacional(Restriccion):
    @abstractmethod
    def permite(self, sesion, franja, aula, estado) -> bool: ...

    def afecta(self, franja, instancia):
        """Franjas de otras sesiones que pueden quedar inválidas al ocupar `franja`.

        Por defecto solo la misma franja (cruces). Una regla con mayor alcance
        lo declara aquí, y el motor poda esas franjas sin conocer la regla.
        """
        return (franja.id,)

    def relacionadas(self, sesion, pendientes):
        """Sesiones pendientes que esta regla puede afectar al asignar `sesion`.

        Por defecto todas; una regla que solo relaciona ciertos recursos lo restringe
        para que la poda no revise sesiones que no comparten nada con la asignada.
        """
        return pendientes
```

```python
def _propagar(self, s, f):
    """Forward checking por regla: cada regla poda solo sus franjas y sesiones relacionadas."""
    podas = []
    pendientes = [t for t in self.sesiones if t.id not in self.estado.asignacion]
    for r in self.relacionales:
        franjas = r.afecta(f, self.inst)
        for t in r.relacionadas(s, pendientes):
            dt = self.dom[t.id]
            for fid in franjas:
                aulas = dt.get(fid)
                if not aulas:
                    continue
                ft = self.franja[fid]
                for aid in list(aulas):
                    if not r.permite(t, ft, self.aula[aid], self.estado):
                        aulas.discard(aid)
                        podas.append((t.id, fid, aid))
                if not aulas:
                    del dt[fid]
            if not dt:
                self.culpables[(r.id, r.recurso(t))] += 1
                for tid, pf, pa in podas:
                    self.dom[tid].setdefault(pf, set()).add(pa)
                return [], False
    return podas, True
```

La prueba de cambio realizada es que la Facultad decide que la preferencia de no programar una asignatura en días consecutivos pase a ser obligatoria cuando tiene dos o tres clases semanales. La Tabla 5 resume qué se modificó, según el historial del repositorio. Después del cambio, todos los horarios siguieron completos y sin incumplimientos, y ninguno quedó con clases en días consecutivos (antes había 0,3 casos por horario en promedio).

Tabla 5

Elementos modificados y no modificados al volver obligatoria la regla de días no consecutivos

Elemento del diseño	¿Cambia?	Detalle
DiasNoConsecutivos (nueva clase)	Sí	Archivo nuevo de 26 líneas
RegistroRestricciones	Sí	Dos líneas para activar la regla
Verificador de pruebas	Sí	Ocho líneas: debe comprobar la nueva regla
BacktrackingSolver, Estado, Editor, Instancia, ValidadorEntrada	No	Ninguna línea. El editor rechaza los cambios que rompen la nueva regla sin modificarse
Interfaz web	Mínimo	Agregar el mensaje de rechazo de la nueva regla

La prueba reveló que el cambio funcionó sin tocar el motor, pero con el doble de datos el tiempo subió de 0,74 s a 8,89 s, por encima del máximo de 5 s. Se agregó a la abstracción el método relacionadas(), un cambio único al motor, y bajó a 0,57 s. Además, bajar el tope a 35 estudiantes es solo un dato de Parametros (el sistema marca 9 secciones que lo superan), pero mover la hora de receso exige editar el código: las franjas aún no son configurables.

G. Escalabilidad

Tabla 6

Tiempo de generación al duplicar y cuadruplicar docentes, aulas y cursos

Tamaño	Docentes	Aulas	Secciones	Clases	Versión inicial: promedio (s)	Versión mejorada: promedio y máximo (s)	Horarios completos
Actual	12	8	44	84	0,14	0,12 y 0,13	10 de 10
Doble	24	16	88	168	0,74	0,51 y 0,56	10 de 10
Cuádruple	48	32	176	336	7,17	5,24 y 30,05	9 de 10

Primer cuello de botella, paso a paso**.** (1) Cada vez que el motor ubica una clase, revisa las opciones de las clases pendientes que podrían chocar con ella. Por la regla de aulas, todas pueden chocar, porque cualquiera podría querer la misma aula. Ese trabajo crece con las clases por las aulas y se repite por cada clase, así que duplicar ambos podría multiplicarlo hasta por 8;

se midió 5,1 veces (versión inicial) y 4,4 (mejorada). El cuello de botella es esa revisión de opciones (BacktrackingSolver._propagar).

La consecuencia es que con el doble de datos el tiempo sigue muy por debajo de 5 s (máximo 0,56 s). Con el cuádruple, la mitad de los casos tarda 2,47 s o menos, pero uno de diez no terminó en 30 s. Si los cursos crecen y las aulas no, el límite es físico: con 66 % de ocupación el horario sale en menos de 0,5 s; con 82 %, no sale en 30 s. Para 2.3 se propone una tabla de ocupación por aula y reintentos con otro orden de búsqueda.

H. Primer Borrador de Diagramas

La Figura 1 es el diagrama de clases del núcleo y la Figura 2, un diagrama de secuencia. Se eligió el diagrama de secuencia porque resolver un conflicto es el supuesto más débil del boceto: muestra quién responde cada paso y que el editor usa las mismas reglas que el motor.

Figura 1

Diagrama de clases del núcleo del sistema (borrador)

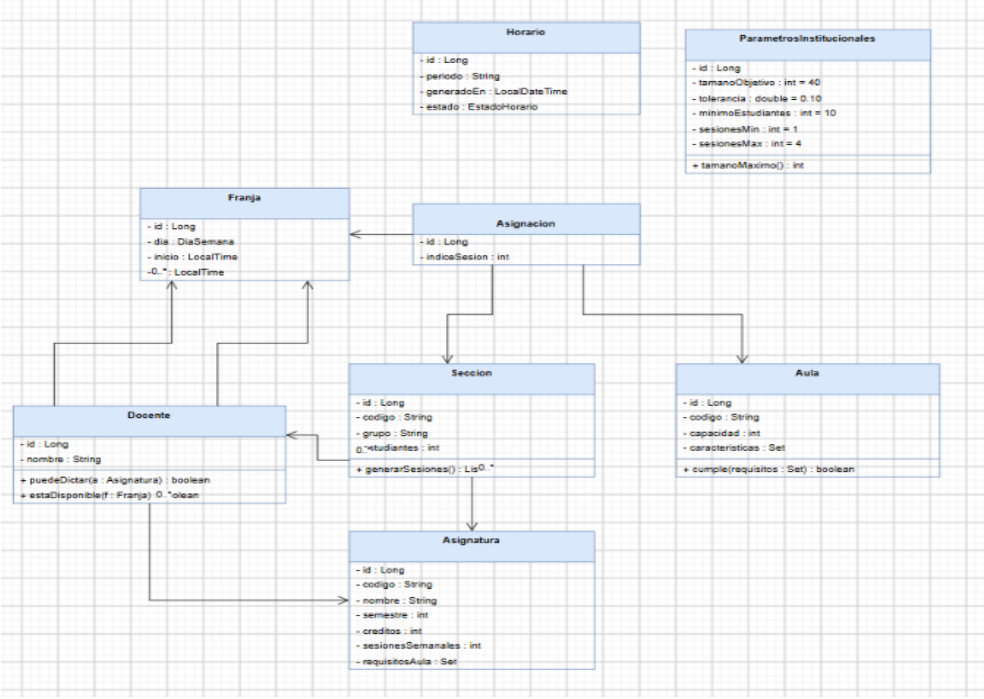
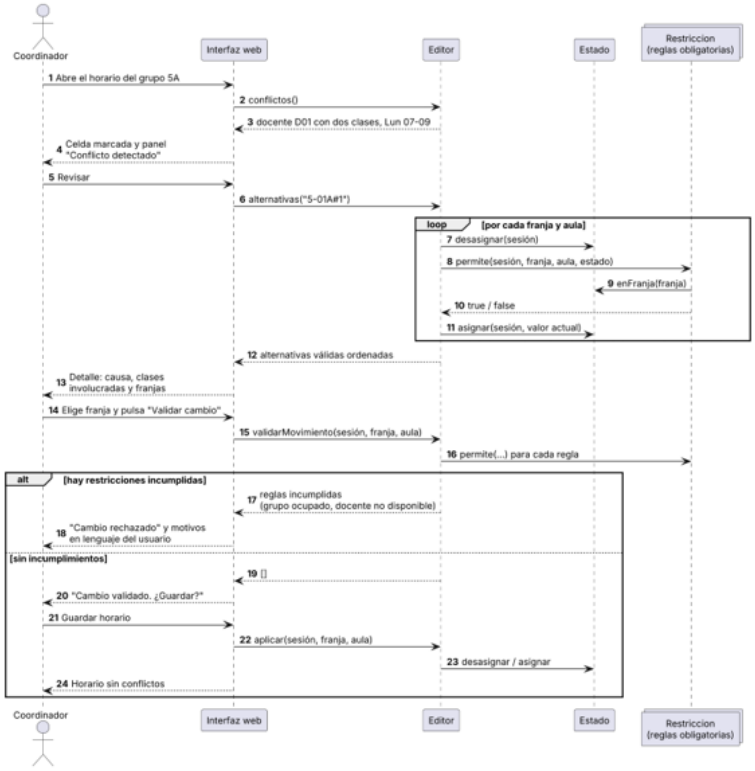


Figura 2

Diagrama de secuencia: detección, revisión y solución de un conflicto de horario



I. Qué Cambió y Ruta hacia el Entregable 2.3

Tabla 7

Supuestos del M1 y de la Fase 1 contrastados con la evidencia

Supuesto	Origen	Estado	Evidencia o razón
La búsqueda con retroceso genera horarios completos en menos de 5 s	Entregable 1	Confirmado	10 de 10 en tamaño actual y doble, sin incumplimientos, en menos de 0,8 s
Elegir primero la clase con menos opciones y descartar opciones bloqueadas es necesario	Entregable 1	Confirmado con matiz	En el tamaño actual no hace falta; con el doble de cursos y las mismas aulas, sin esas técnicas no se termina en 30 s
La búsqueda explica qué regla y qué elementos impiden el horario	Entregable 1	Modificado	Culpó a un docente que no era la causa; la explicación correcta la da la revisión previa mejorada
Strategy para el motor de generación	Fase 1	Confirmado con ajuste	Sirve para cambiar de método; las variantes del retroceso bastan con parámetros
Una regla nueva no exige tocar el motor	Entregable 1	Confirmado con ajuste	Cierto, pero hubo que corregir una pérdida de rendimiento
Franjas y topes configurables sin tocar código	Entregable 1	Parcial	El tope de estudiantes sí; las franjas siguen en el código
El motor asigna los docentes	Entregable 1	Descartado	El MVP recibe el docente de cada sección y solo verifica que esté habilitado
Edición manual arrastrando clases	Entregable 1	Descartado para el MVP	Se usa una lista de franjas con validación inmediata, como en el boceto
El sistema valida un cambio y propone alternativas al instante	Boceto	Confirmado	Validación en 0,05 ms como máximo; alternativas en 3 a 12 ms
El coordinador entiende la causa y resuelve sin ayuda	Boceto	Confirmado por inspección	La versión 2 no tuvo fallas en el recorrido cognitivo
Medir la interacción con tres participantes	Fase 1	Modificado	Se reemplazó por métodos de inspección que no requieren participantes
Un grupo es una cohorte que toma todas las asignaturas de su semestre	Fase 1	Provisional	Falta confirmarlo con el Programa

Tabla 8

Avance hacia cada parte del entregable 2.3

Parte de 2.3	Decisión	Evidencia	Responsable	Estado y qué falta
Contextualización	Documentar los cambios de la Tabla 7	Este dossier	L. N. Olmos	Borrador. Falta corregir en el Entregable 1 los requisitos de asignar docentes y de arrastrar clases
Arquitectura	Capas de dominio, reglas, motor, edición, interfaz y persistencia	Prototipo técnico	L. V. Ospina	Borrador. Falta elegir la tecnología web y la base de datos
Modelo de datos	Entidades de la Figura 1; docente fijo por sección	modelo.py	L. Palacios	Borrador. Falta confirmar datos reales con el Programa
Algoritmo	Búsqueda con retroceso y revisión previa mejorada	Tablas 1 y 6	L. N. Olmos y L. Palacios	Decidido para el MVP. Falta probar reintentos con otro orden
Patrón de diseño	Strategy en el motor	Tabla 4	L. V. Ospina y F. Mejía	Decidido. Falta implementar ServicioProgramacion
SOLID y modificabilidad	Abierto/cerrado e inversión de dependencias en las reglas	Tabla 5	F. Mejía	Decidido. Falta volver configurables las franjas
Atributos de calidad	Escala hasta el doble con margen	Tabla 6	L. Palacios	Borrador. Falta medir con datos reales
Mockups	Franjas válidas sugeridas primero (hecho en la versión 2)	Tablas 2 y 3; Anexo C	L. V. Ospina y F. Mejía	Borrador. Faltan un solo botón para aplicar y las pantallas de carga de datos
Diagramas	Figuras 1 y 2	Repositorio	L. V. Ospina y L. Palacios	Borrador. Falta añadir persistencia e interfaz
Trazabilidad	Tabla 9	Tabla 9	Todo el equipo	Borrador. Faltan las demás reglas y preferencias del Entregable 1

Tabla 9

Esqueleto de la matriz de trazabilidad

Regla del M1	Requisito que la atiende	Componente del diseño	Cómo se verifica
Un aula no puede tener dos clases a la vez	Generar y validar el horario; validar cambios manuales	AulaLibre; BacktrackingSolver; Editor	Verificador: 0 incumplimientos

Un docente no puede tener dos clases a la vez	Generar y validar el horario; validar cambios manuales	DocenteLibre; ValidadorEntrada	Verificador; caso del docente sin franjas
Un grupo no puede tener dos asignaturas a la vez	Generar y validar el horario; reportar conflictos	GrupoLibre; ValidadorEntrada	Verificador; caso del grupo 7A
El aula debe tener el equipamiento requerido	Registrar requisitos de cada asignatura y de las aulas	CaracteristicasAula; ValidadorEntrada	Caso de una sola sala de cómputo
Respetar la disponibilidad de cada docente	Registrar disponibilidad de docentes	DisponibilidadDocente	Verificador; caso del docente sin franjas
Tamaño máximo de sección	Configurar tamaño y tolerancia	Parametros; ValidadorEntrada	Prueba del tope de 35
Parámetros institucionales configurables	Registrar y modificar franjas	Parametros (franjas pendientes)	Prueba de cambio (parcial)
Preferencia de días no consecutivos	Evaluar las preferencias	EvaluadorBlandas; orden de búsqueda	0,3 casos por horario

Referencias

Card, S. K., Moran, T. P., & Newell, A. (1980). The keystroke-level model for user performance time with interactive systems. *Communications of the ACM, 23*(7), 396–410.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Hu, C. (2023). *An introduction to software design: Concepts, principles, methodologies, and techniques*. Springer.

López-Cruz, O. (s. f.). *Some notes on prototypes* [Material de clase]. Universidad El Bosque.

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 152–158). ACM.

Ortega-Arjona, J. L. (2010). Software patterns. En *Patterns for parallel software design* (cap. 1). Wiley.

Russell, S., & Norvig, P. (2022). *Artificial intelligence: A modern approach* (4.ª ed. global). Pearson Education.

Wharton, C., Rieman, J., Lewis, C., & Polson, P. (1994). The cognitive walkthrough method: A practitioner's guide. En J. Nielsen & R. L. Mack (Eds.), *Usability inspection methods* (pp. 105–140). Wiley.

Anexo A. Evidencia del Prototipo Técnico

Para reproducir la carpeta del prototipo se ejecuta el comando: python prototipo_visual.py en la terminal y automáticamente en el navegador se abrirá el prototipo.

Tabla 10

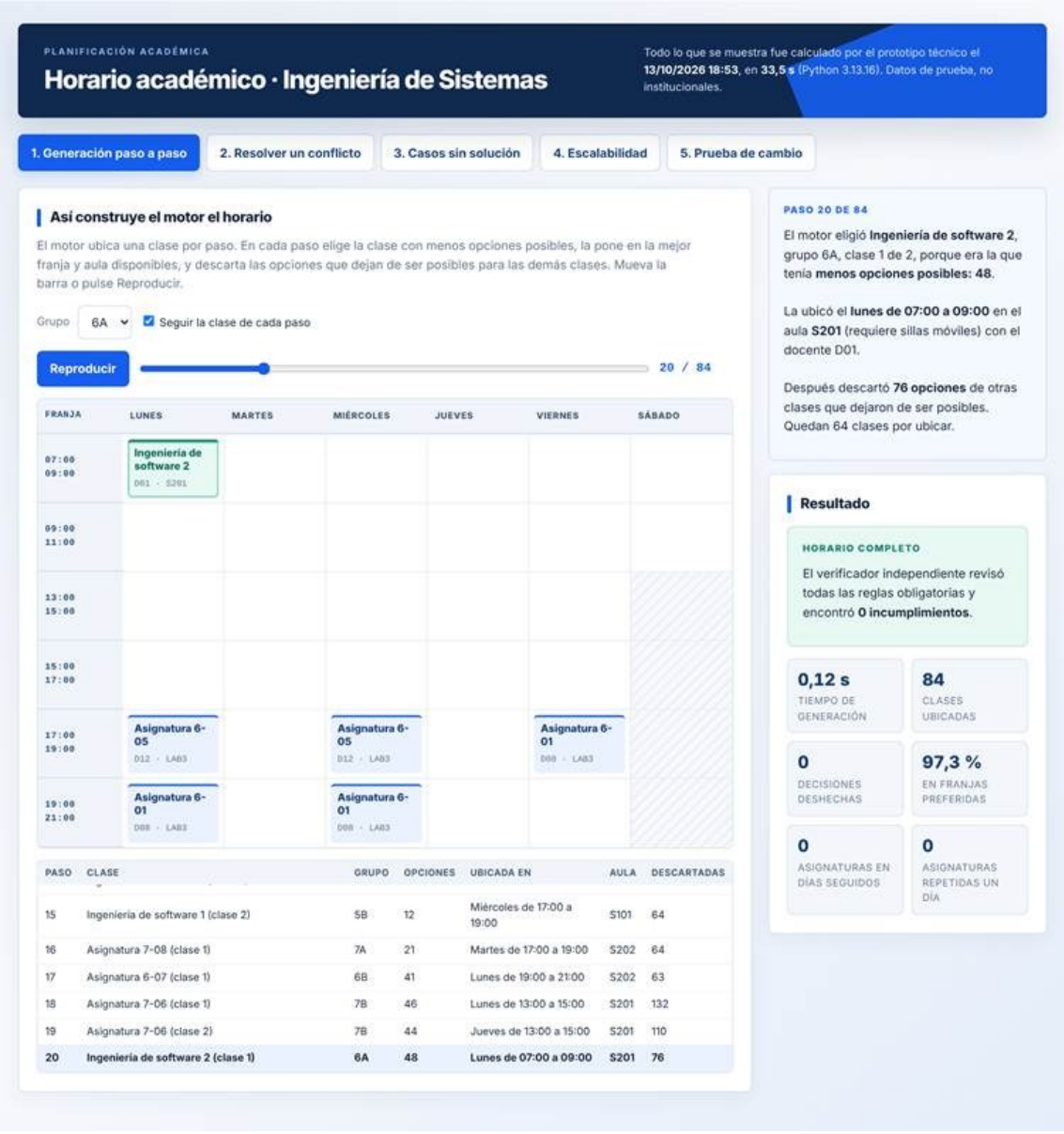
Corridas del prototipo técnico

Corrida	Qué mide	Resultado
E1	Horarios posibles en tamaño actual, doble y cuádruple; 10 casos cada uno	Actual: 10 de 10, 0,14 s. Doble: 10 de 10, 0,74 s. Cuádruple: 9 de 10. Ningún incumplimiento. El 96,5 % de las clases quedó en franjas preferidas por el docente
E1b	Crecen cursos y docentes, pero no las aulas	Con 66 % de aulas ocupadas sale el horario; con 98 %, no sale en 30 s
E2	Búsqueda simple, sin técnicas de orden ni descarte, en tamaño actual	Resuelve los 5 casos: en tamaño actual el problema es holgado (33 % de ocupación)
E2b	Con y sin técnicas de orden y descarte, aulas fijas	Doble (66 %): con técnicas, entre 0,33 y 0,45 s; sin ellas, no termina en 30 s. Con 82 %: ninguna termina en 30 s
E3	Casos imposibles	Tabla 1
E4	Validación de 500 cambios manuales y alternativas de 50 clases	Validación: 0,01 ms en promedio; 84,8 % de cambios al azar rechazados. Alternativas: 2,9 ms (actual) y 7,6 ms (doble); unas 23 válidas por clase
E5	Dónde se gasta el tiempo	Revisión de opciones del motor: 31,5 %, 48,8 % y 43,6 % del tiempo en la versión inicial; preparación de opciones iniciales: entre 28 % y 45 %

Cambio	Regla de días no consecutivos obligatoria	Antes de la mejora: 0,93 s (actual) y 8,89 s (doble). Después: 0,12 s y 0,57 s. Todos completos y sin incumplimientos

Figura 3

Prototipo visual: construcción del horario paso a paso



Nota. En el paso 20 el motor eligió la clase con menos opciones, la ubicó y descartó las opciones que dejaron de ser posibles. A la derecha, el resultado final: 0 incumplimientos.

Figura 4

Prototipo visual: casos en los que no existe un horario

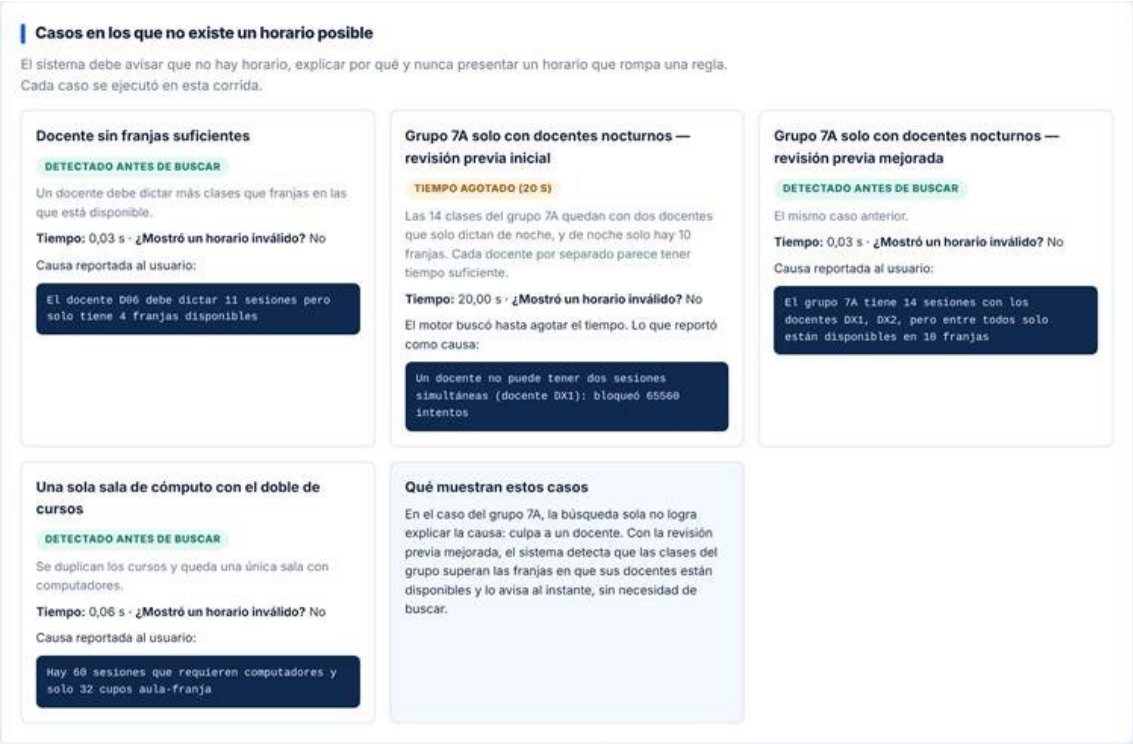


Figura 5

Prototipo visual: tiempos al duplicar docentes, aulas y cursos



Supuestos de los datos de prueba. Clases semanales según créditos (4 créditos, 3 clases; 3 o 2 créditos, 2; 1 crédito, 1); siete asignaturas requieren computadores y dos, sillas móviles; secciones de 18 a 44 estudiantes; uno de cada cuatro docentes solo dicta desde las 5:00 p. m. y los demás tienen entre 4 y 8 franjas bloqueadas. Los nombres de asignaturas del prototipo de interacción son ilustrativos.

Anexo B. Prueba de Cambio: Código de la Nueva Regla

```python
from .restricciones import RestriccionRelacional


class DiasNoConsecutivos(RestriccionRelacional):
    id = "DIAS-NO-CONSECUTIVOS"
    descripcion = "Las sesiones de una sección no pueden quedar el mismo día ni en días consecutivos"

    def permite(self, sesion, franja, aula, estado):
        if sesion.seccion.asignatura.sesiones_semanales not in (2, 3):
            return True
        for dia in (franja.dia - 1, franja.dia, franja.dia + 1):
            for otra, _, _ in estado.en_dia(dia):
                if otra.seccion.id == sesion.seccion.id and otra.id != sesion.id:
                    return False
        return True

    def afecta(self, franja, instancia):
        return tuple(f.id for f in instancia.franjas if abs(f.dia - franja.dia) <= 1)

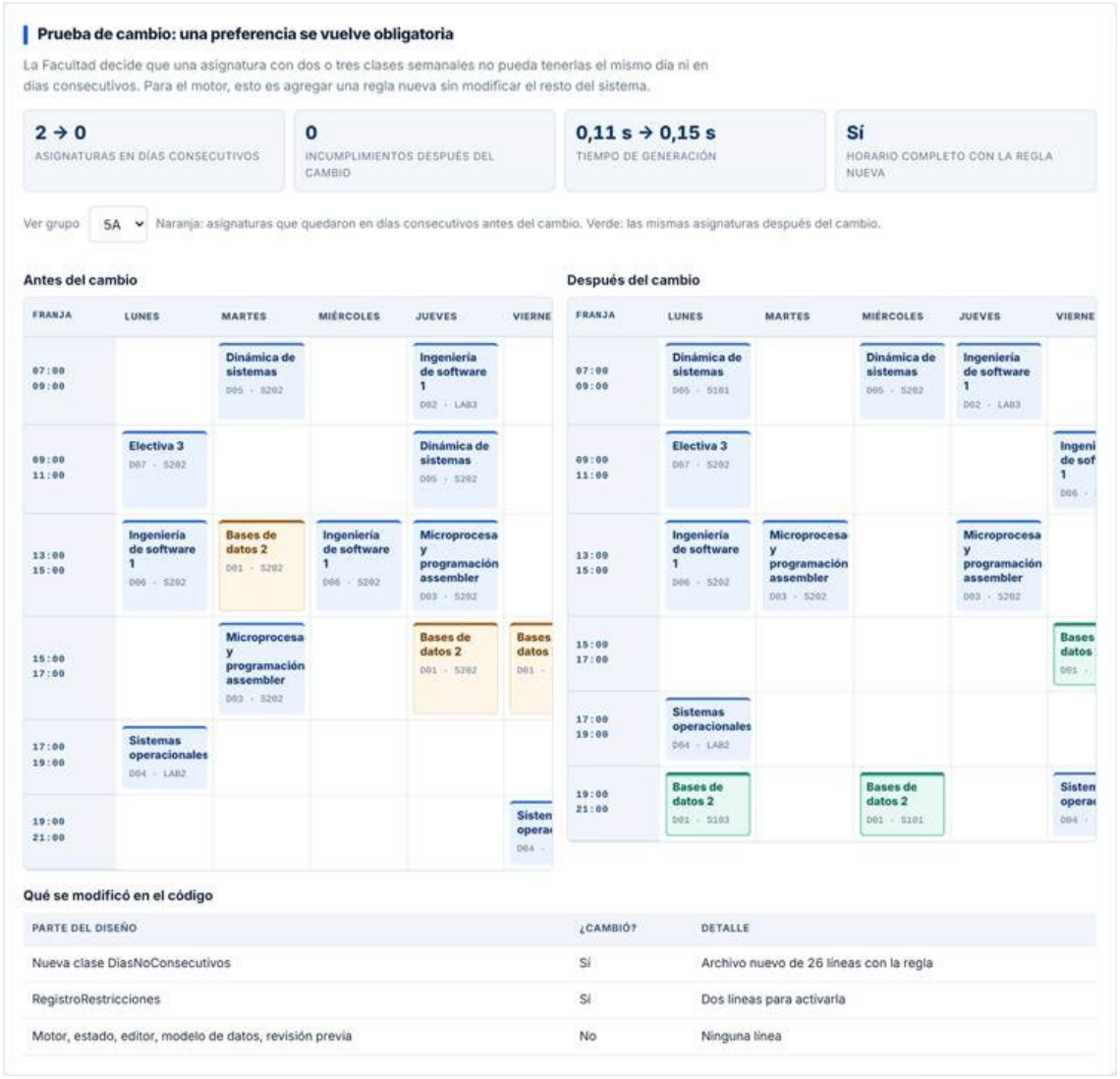
    def relacionadas(self, sesion, pendientes):
        return [t for t in pendientes if t.seccion.id == sesion.seccion.id]

    def recurso(self, sesion):
        return f"sección {sesion.seccion.id}"

```

Figura 6

Prototipo visual: horario antes y después de volver obligatoria la regla



Ajuste posterior al motor el método relacionadas() en la abstracción, con una versión por defecto que las reglas de docente, grupo y días no consecutivos reemplazan, y la revisión de opciones reescrita para trabajar regla por regla.

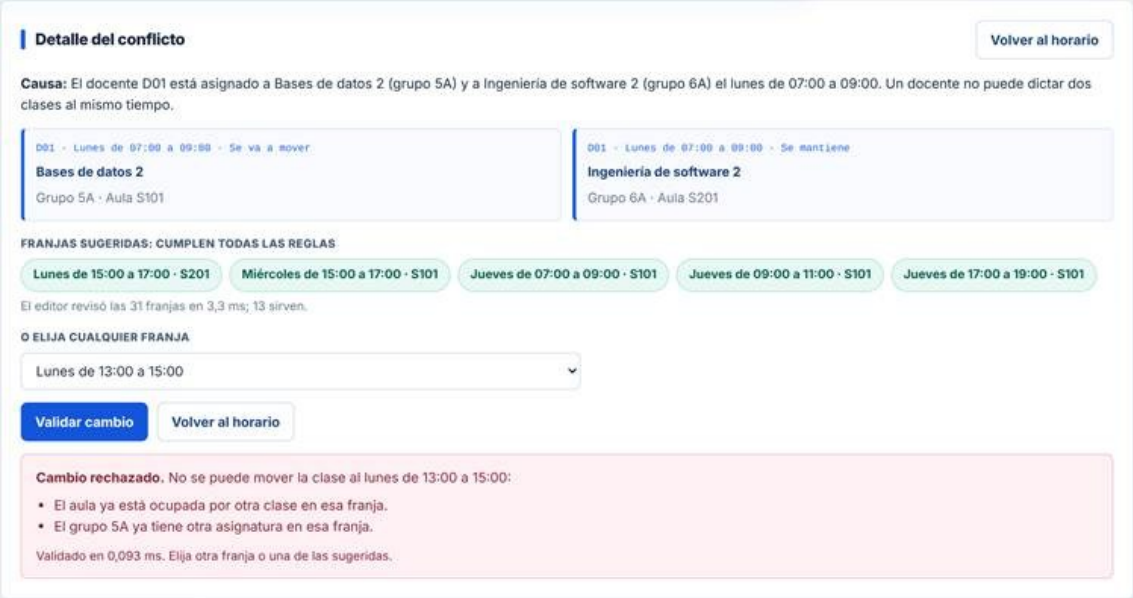
Anexo C. Evidencia del Prototipo de Interacción

Escenario evaluado. Horario del grupo 5A con un cruce real del docente D01: dos clases el lunes de 7:00 a 9:00 a. m. La tarea es dejar el horario sin conflictos.

Procedimiento de inspección. (1) Recorrido cognitivo: en cada paso se respondieron las cuatro preguntas del método (si el usuario intentará el efecto correcto, si notará la acción disponible, si la asociará con ese efecto y si verá el progreso). Evaluación heurística: cada pantalla se revisó con las diez heurísticas de Nielsen, con gravedad de 0 (no es problema) a 4 (catastrófico). Tiempo: se sumaron los operadores del modelo de pulsaciones. En la versión 1, con 13 franjas válidas entre 31, elegir al azar produce en promedio 18 ÷ 14 = 1,3 rechazos de unos 14 s cada uno. En la versión 2, elegir una sugerencia cuesta preparación mental, lectura de cinco opciones y un clic (10,2 s), sin rechazos.

Figura 7

Versión 2 dentro del prototipo visual: franjas válidas sugeridas primero



Anexo D. Bitácora de Trabajo

La Tabla 12 compara el plan de la Fase 1 con lo ocurrido.

Tabla 11

Plan de la Fase 1 frente a lo ocurrido

Fecha planeada	Actividad planeada	Responsables	Qué ocurrió	Evidencia
12 oct	Revisar preguntas, criterios y escenarios de prueba	Todo el equipo	Se solucionaron dudas y se definió como se quería llevar a cabo el prototipo y la fase 2.	Respuestas a preguntas de la Fase 1 a lo largo del documento.
12-13 oct	Prototipo técnico y escenarios posibles e imposibles	L. N. Olmos, L. Palacios	Construido el 13 oct con asistencia de IA.	Boceto fase 1, aplicativo en pyhton
12-13 oct	Prototipo de interacción y prueba	L. V. Ospina, F. Mejía	Prototipo construido el 13 oct con asistencia de IA. La prueba con tres personas se reemplazó por inspección	Tablas 2 y 3; Anexo C
13-14 oct	Tiempos y escalabilidad	L. N. Olmos, L. Palacios	Corridas E1 a E5 el 13 oct	Carpeta resultados
13-14 oct	Revisión de Strategy y borrador de clases	L. V. Ospina, F. Mejía	Ficha y diagrama en borrador el 13 oct.	Tabla 4; Figura 1
14 oct	Diagramas	L. V. Ospina, L. Palacios	Adelantados al 13 oct	Carpeta diagramas
14 oct	SOLID y prueba de cambio	Todo el equipo	Prueba realizada el 13 oct	
14-15 oct	Trazabilidad y tabla de avance	Todo el equipo	Borrador el 13 oct.	Tablas 8 y 9

Anexo E. Memorias Individuales

Laura Natalia Olmos Parra

Aporte concreto y verificable: Participé, junto con mi compañera Laura Palacios, en el desarrollo del prototipo técnico de generación de horarios: escenarios con y sin solución, comprobación restricciones obligatorias, registrar los tiempos de ejecución y analizar qué ocurre al duplicar los datos.

Una decisión que defendería: Separar el diseño en paquetes (controller, service, repository y model), de modo que cada capa solo dependa de la de abajo. Así, cambiar la base de datos o la interfaz web no obliga a modificar la lógica del horario.

Una pregunta técnica que quedó abierta: ¿Cómo modelar a los estudiantes que cursan asignaturas de varios semestres? El diseño actual asume que cada grupo toma solo las asignaturas de su semestre.

Laura Valentina Ospina Rengifo

Aporte concreto y verificable: Participé en el diseño del prototipo de interacción con el usuario para la resolución de conflictos de horario junto con Felipe Mejia. A partir de su evaluación se identificó que mostrar todas las franjas disponibles sin distinguir cuales eran válidas podría llevar al usuario a probar alternativas al azar. Por ello se planteó una segunda versión que presenta primero las franjas válidas. También participe en la formulación de la hipótesis del patrón Strategy para el motor de generación y en el desarrollo del diseño preliminar del sistema.

Una decisión que defendería: Defendería el uso del patrón Strategy para separar el motor de generación de la lógica que coordina la programación. Esta decisión permitiría incorporara o comparar diferentes algoritmos sin modificar innecesariamente los componentes que gestiona los datos o presentan los horarios. Considero importante mantener esta posibilidad de evolución, siempre que su beneficio justifique la complejidad adicional.

Una pregunta técnica que quedó abierta: ¿Cómo debería definirse la interfaz común del motor de generación para permitir sustituir el algoritmo de backtracking por otro, como CP-SAT, sin perder información sobre los conflictos, las causas por las que no se encontró una solución o las limitaciones de cada estrategia?

Laura Palacios Ochoa

Aporte concreto y verificable: Participé en desarrollo y pruebas del prototipo, evaluando restricciones duras, tiempos de ejecución y escalabilidad. Además, ayudé con la revisión de trazabilidad entre requisitos, diseño y pruebas.

Una decisión que defendería: Mantener un alcance reducido del motor de horarios, priorizando que cumpla correctamente con las restricciones seleccionadas antes de aumentar la complejidad del problema.

Una pregunta técnica que quedó abierta: ¿Hasta qué volumen de datos puede escalar nuestro algoritmo manteniendo tiempos de ejecución aceptables?

Felipe Mejía Alvarado

Aporte concreto y verificable: Participé en la construcción del prototipo de interacción y en su evaluación a través de métodos de inspección. También ayudé en la elaboración del patrón Strategy y borrador de las clases del sistema.

Una decisión que defendería: Mostrar primero las franjas horarias válidas al momento de resolver un conflicto, con el fin de facilitar la elección de una nueva alternativa y así permitir al usuario completar la tarea sin errores.

Una pregunta técnica que quedó abierta: ¿Cómo volver configurables las franjas y la hora del receso sin romper el resto del diseño? Actualmente se encuentran fijas, por lo que mover el receso exige editar código. Queda por definir si las franjas deben cargarse desde la base de datos y cómo afectaría eso a las secciones que ya tienen un horario asignado.

Anexo F. Autoevaluación del Equipo

Los conocimientos de Ingeniería de Software de semestres anteriores facilitaron esta entrega, porque permitieron plantear las restricciones como requisitos verificables, construir la matriz de trazabilidad y elaborar los diagramas de clases y de secuencia con notación UML. Los cursos de programación también fueron determinantes para organizar el trabajo: la programación orientada a objetos permitió separar el modelo de datos, las reglas y el motor en clases con responsabilidades claras, y las bases de estructuras de datos y algoritmos ayudaron a entender cómo el motor recorre las opciones y por qué su tiempo crece con la cantidad de datos. En cambio, el equipo considera que le falta profundidad en los algoritmos de optimización combinatoria, temas que serán necesarios si el sistema debe atender más semestres o aulas más ocupadas.

Anexo G. Transparencia de Uso de IA

Herramienta. Claude (Anthropic), modelo Claude Opus 5.5

Usos en la Fase 2. (1) Asistencia en la escritura del código para el prototipo en Python (2) Redacción del borrador de este dossier, sus tablas y la ficha de patrón.